

VOLUME 5

BLOCO A

ARQUITETURA, ENGENHARIA DE SOFTWARE E IMPLANTAÇÃO

BASE OFICIAL DE CONHECIMENTO

AGENTE AURORA

RAG EDITION – REVISÃO FINAL

CONTROLE DE REVISÕES

Versão	Data	Responsável	Descrição
1.0	Agosto/2026	Projeto Aurora	Primeira versão da documentação técnica
2.0	Agosto/2026	Projeto Aurora	Revisão completa da arquitetura, alinhada ao Desafio Final da Alura

APRESENTAÇÃO

Este volume descreve toda a arquitetura técnica utilizada na implementação do **Agente Aurora**, um recepcionista virtual inteligente desenvolvido como projeto do **Challenge Alura Agente – Tech AI Builder**.

Diferentemente dos volumes anteriores, que documentam o domínio de negócio da pousada e constituem a Base Oficial de Conhecimento utilizada pelo sistema RAG, este documento estabelece os princípios técnicos que orientarão a implementação da solução.

Seu propósito é garantir que todos os componentes da aplicação sejam desenvolvidos de forma organizada, modular e preparada para futuras evoluções, preservando a simplicidade necessária para a entrega do desafio.

CAPÍTULO 1

OBJETIVO DA ARQUITETURA

1.1 Finalidade

A arquitetura do Agente Aurora foi projetada para atender aos seguintes objetivos:

- responder perguntas utilizando uma Base Oficial de Conhecimento;
- reduzir alucinações por meio de Retrieval-Augmented Generation (RAG);
- executar integralmente em ambiente local durante o desenvolvimento;
- utilizar ferramentas gratuitas e de código aberto;
- facilitar manutenção e evolução do sistema;
- permitir substituição do modelo de linguagem sem alterações significativas na aplicação.

1.2 Princípios Arquiteturais

A solução foi concebida com base nos seguintes princípios:

- simplicidade;
- modularidade;
- reutilização de componentes;
- baixo acoplamento;
- separação entre conhecimento e código;
- escalabilidade futura;
- independência do provedor de IA.

Esses princípios orientam todas as decisões de projeto.

CAPÍTULO 2

ESCOPO DO PROJETO

O Agente Aurora é um recepcionista virtual especializado no atendimento aos hóspedes da Pousada Mirante do Pôr do Sol.

Entre suas principais responsabilidades estão:

- apresentar acomodações;
- explicar serviços oferecidos;
- responder perguntas frequentes;
- informar políticas da pousada;
- sugerir passeios turísticos;
- orientar hóspedes durante o processo de hospedagem;
- registrar intenção de reserva (simulada).

Nesta primeira versão, o sistema operará exclusivamente por meio de uma interface web desenvolvida em Streamlit.

CAPÍTULO 3

STACK TECNOLÓGICO OFICIAL

A implementação utilizará exclusivamente as tecnologias abaixo.

Camada	Tecnologia
Linguagem	Python
Prototipação	Google Colab

Camada	Tecnologia
Desenvolvimento	VS Code
Framework de IA	LangChain
Modelo de Linguagem	Ollama
Embeddings	Ollama Embeddings
Banco Vetorial	FAISS
Interface	Streamlit
Versionamento	GitHub
Deploy	Oracle Cloud Infrastructure (OCI)

Este conjunto foi definido por equilibrar simplicidade, baixo custo e aderência aos requisitos do desafio.

CAPÍTULO 4

TECNOLOGIAS FORA DO ESCOPO

Para evitar ambiguidades durante a implementação, ficam explicitamente excluídas desta versão do projeto as seguintes tecnologias:

- n8n;
- Docker;
- PostgreSQL;
- pgvector;
- Telegram;
- WhatsApp;
- OpenAI API;
- Google Gemini API;
- Anthropic Claude API;
- bancos vetoriais em nuvem.

Essas tecnologias poderão ser avaliadas em versões futuras, mas não fazem parte da entrega atual.

CAPÍTULO 5

VISÃO GERAL DA SOLUÇÃO

O funcionamento do sistema está dividido em duas grandes etapas:

Construção da Base Vetorial

Realizada apenas quando a documentação oficial é criada ou atualizada.

Fluxo:

1. Leitura dos documentos PDF.
 2. Extração do texto.
 3. Fragmentação em chunks.
 4. Geração dos embeddings.
 5. Construção do índice vetorial FAISS.
 6. Persistência do índice.
-

Atendimento ao Usuário

Executado continuamente durante a utilização da aplicação.

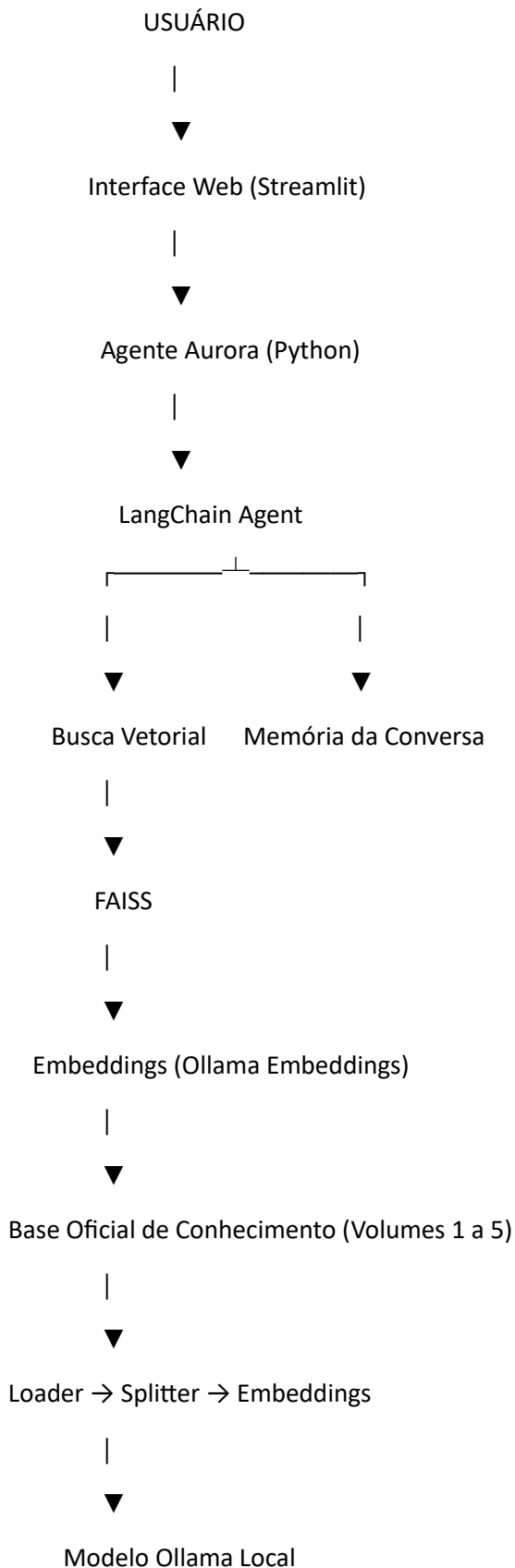
Fluxo:

1. Usuário envia uma pergunta.
 2. Streamlit encaminha a solicitação ao Agente Aurora.
 3. O LangChain realiza consulta ao índice FAISS.
 4. Os trechos mais relevantes são recuperados.
 5. O contexto é enviado ao modelo Ollama.
 6. O modelo gera a resposta.
 7. A resposta é apresentada ao usuário.
-

CAPÍTULO 6

ARQUITETURA OFICIAL DO AGENTE AURORA

A arquitetura lógica do sistema é representada pela seguinte estrutura:



Esta arquitetura representa a implementação oficial da primeira versão do projeto.

CAPÍTULO 7

BASE OFICIAL DE CONHECIMENTO

A Base Oficial de Conhecimento é composta pelos cinco volumes produzidos durante a fase documental do projeto.

Esses documentos constituem a única fonte autorizada de informação utilizada pelo sistema RAG.

Sua função é fornecer contexto para a geração das respostas, garantindo consistência e reduzindo a probabilidade de informações incorretas.

Estrutura da Base

Volume	Conteúdo
--------	----------

Volume 1	Institucional
----------	---------------

Volume 2	Acomodações
----------	-------------

Volume 3	Serviços e Políticas
----------	----------------------

Volume 4	Turismo e Concierge
----------	---------------------

Volume 5	Arquitetura e Engenharia de Software
----------	--------------------------------------

Cada volume possui organização padronizada, facilitando a leitura, a fragmentação em chunks e a recuperação semântica.

CAPÍTULO 8

REQUISITOS FUNCIONAIS

O Agente Aurora deverá ser capaz de:

- responder perguntas institucionais;
 - localizar informações sobre acomodações;
 - informar preços e benefícios cadastrados na Base Oficial;
 - responder dúvidas sobre serviços;
 - explicar políticas de reserva, check-in e cancelamento;
 - sugerir passeios em Campos do Jordão;
 - consultar informações por similaridade semântica;
 - registrar intenção de reserva em banco de dados (simulado).
-

CAPÍTULO 9

REQUISITOS NÃO FUNCIONAIS

O sistema deverá atender aos seguintes requisitos:

- interface simples e intuitiva;
- respostas em linguagem natural;
- arquitetura modular;
- código organizado por responsabilidade;
- facilidade de manutenção;
- independência do provedor de IA;
- execução em ambiente local durante o desenvolvimento;
- preparação para implantação na Oracle Cloud Infrastructure.

CAPÍTULO 10

OBJETIVOS DA ENGENHARIA DE SOFTWARE

A fase de engenharia de software terá como objetivos:

- transformar a Base Oficial de Conhecimento em uma base vetorial pesquisável;
- implementar o pipeline RAG em Python;
- construir uma aplicação web utilizando Streamlit;
- integrar LangChain, FAISS e Ollama;
- organizar o código em módulos reutilizáveis;
- preparar o projeto para deploy na Oracle Cloud Infrastructure.

RESUMO EXECUTIVO DO BLOCO A

Este bloco estabelece a arquitetura oficial do Agente Aurora, definindo seu escopo, objetivos, princípios arquiteturais e tecnologias adotadas. Também formaliza a separação entre a Base Oficial de Conhecimento e a implementação em software, apresenta a arquitetura lógica do sistema, descreve o fluxo geral de funcionamento e consolida os requisitos funcionais e não funcionais que orientarão todas as etapas de desenvolvimento.

ENCERRAMENTO DO BLOCO A

VOLUME 5

BLOCO B

ARQUITETURA, ENGENHARIA DE SOFTWARE E IMPLANTAÇÃO

CAPÍTULO 11

PIPELINE RAG

11.1 Objetivo

O pipeline RAG (Retrieval-Augmented Generation) é o núcleo do Agente Aurora.

Sua função é permitir que o modelo de linguagem responda perguntas utilizando a Base Oficial de Conhecimento em vez de depender exclusivamente do conhecimento pré-treinado do modelo.

Essa arquitetura reduz significativamente a ocorrência de respostas incorretas, melhora a precisão das informações e facilita a atualização do conhecimento sem necessidade de novo treinamento do modelo.

11.2 Componentes do Pipeline

O pipeline será composto pelas seguintes etapas:

1. Base Oficial de Conhecimento
2. Loader
3. Text Splitter
4. Geração de Embeddings
5. Índice Vetorial (FAISS)
6. Recuperação Semântica
7. LangChain
8. Modelo Ollama
9. Resposta ao Usuário

Cada componente possui responsabilidade única, favorecendo modularidade e manutenção.

CAPÍTULO 12

LEITURA DA BASE DE CONHECIMENTO

A Base Oficial de Conhecimento é composta pelos cinco volumes produzidos durante a fase documental.

Esses documentos serão armazenados em formato PDF no diretório:

aurora-ai-assistant/

└─ documentos/

└─ Volume_1.pdf
└─ Volume_2.pdf
└─ Volume_3.pdf
└─ Volume_4.pdf
└─ Volume_5.pdf

A leitura será realizada por um componente dedicado (*loader*), responsável por extrair o conteúdo textual preservando a estrutura lógica dos documentos.

Responsabilidades do Loader

- localizar os arquivos PDF;
- carregar o conteúdo textual;
- preservar a ordem dos capítulos;
- encaminhar os documentos para a etapa de fragmentação.

A implementação do *loader* seguirá a abordagem apresentada nos notebooks de referência, adaptada à estrutura documental do projeto.

CAPÍTULO 13

FRAGMENTAÇÃO DOS DOCUMENTOS (CHUNKING)

Objetivo

Os modelos de linguagem trabalham melhor com trechos menores de texto.

Por esse motivo, cada documento será dividido em fragmentos (*chunks*) antes da geração dos embeddings.

Estratégia

A fragmentação deverá preservar o contexto semântico.

Sempre que possível:

- um capítulo permanecerá no mesmo conjunto de chunks;
 - perguntas e respostas da FAQ não serão separadas;
 - listas de políticas permanecerão completas;
 - títulos serão mantidos junto ao conteúdo correspondente.
-

Parâmetros

Os valores exatos serão centralizados no arquivo `config.py`, permitindo ajustes sem alterar a lógica da aplicação.

Entre os parâmetros configuráveis estão:

- tamanho máximo dos chunks;

- sobreposição (*overlap*);
 - diretórios de entrada e saída;
 - modelo de embeddings.
-

CAPÍTULO 14

GERAÇÃO DOS EMBEDDINGS

Após a fragmentação, cada chunk será convertido em uma representação vetorial (*embedding*).

Nesta versão do projeto será utilizado:

- Ollama Embeddings.

A escolha foi motivada por:

- execução local;
 - ausência de custos por requisição;
 - simplicidade de configuração;
 - integração com LangChain.
-

Independência do Modelo

A arquitetura foi concebida para permitir futura substituição do provedor de embeddings.

Assim, componentes como OpenAI, Gemini ou Claude poderão ser incorporados posteriormente com alterações concentradas na camada de configuração, preservando o restante da aplicação.

CAPÍTULO 15

ÍNDICE VETORIAL FAISS

Objetivo

O FAISS será responsável pelo armazenamento e pela pesquisa por similaridade entre os embeddings gerados.

Justificativa

A escolha do FAISS baseia-se em:

- alto desempenho para busca vetorial;
 - integração com LangChain;
 - funcionamento local;
 - facilidade de persistência do índice;
 - ampla adoção em projetos RAG.
-

Organização

O índice será armazenado no diretório:

embeddings/

└─ faiss_index/

Sempre que a Base Oficial for atualizada, um novo índice poderá ser gerado.

CAPÍTULO 16

RECUPERAÇÃO SEMÂNTICA

Durante o atendimento, o sistema não pesquisará palavras exatas.

A recuperação será baseada na similaridade semântica entre a pergunta do usuário e os embeddings armazenados.

Esse mecanismo permite localizar conteúdos relevantes mesmo quando o usuário utiliza expressões diferentes das presentes na documentação.

Fluxo

Pergunta

↓

Embedding da pergunta

↓

Consulta ao FAISS

↓

Recuperação dos chunks mais relevantes

↓

Envio do contexto ao LangChain

CAPÍTULO 17

LANGCHAIN

O LangChain atuará como camada de orquestração da aplicação.

Entre suas responsabilidades estão:

- receber perguntas da interface;
- consultar o índice FAISS;
- montar o contexto recuperado;

- encaminhar o contexto ao modelo Ollama;
 - receber a resposta;
 - devolver o resultado à interface Streamlit.
-

Benefícios

- separação entre componentes;
 - reutilização de código;
 - facilidade para testes;
 - preparação para futuras expansões.
-

CAPÍTULO 18

MODELO DE LINGUAGEM

O Agente Aurora utilizará inicialmente um modelo executado pelo Ollama.

Essa decisão oferece:

- funcionamento totalmente local;
- independência de serviços externos;
- ausência de custos recorrentes;
- maior controle sobre privacidade dos dados.

A escolha do modelo específico poderá ser ajustada conforme desempenho e disponibilidade, sem alterar a arquitetura descrita neste documento.

CAPÍTULO 19

FLUXO COMPLETO DE PROCESSAMENTO

Construção da Base

Volumes PDF



Loader



Text Splitter



Chunks



Embeddings



FAISS



Persistência do Índice

Esta etapa será executada apenas quando houver alterações na Base Oficial de Conhecimento.

Atendimento

Usuário



Streamlit



Aurora



LangChain



FAISS



Recupera os melhores chunks



Ollama



Resposta



Usuário

CAPÍTULO 20

BOAS PRÁTICAS DO PIPELINE

Durante a implementação deverão ser observadas as seguintes diretrizes:

- manter os componentes desacoplados;
- evitar lógica duplicada;
- centralizar configurações;

- documentar parâmetros importantes;
 - preservar a estrutura dos documentos oficiais;
 - utilizar nomes consistentes para módulos e funções;
 - registrar erros de processamento;
 - validar periodicamente a qualidade da recuperação semântica.
-

CrITÉRIOS de Qualidade

O pipeline será considerado adequado quando:

- recuperar informações corretas da Base Oficial;
 - responder utilizando o contexto recuperado;
 - evitar respostas sem fundamentação documental;
 - manter desempenho compatível com atendimento interativo;
 - permitir atualização da Base Oficial sem alterações significativas no código.
-

RESUMO EXECUTIVO DO BLOCO B

Este bloco documenta o pipeline RAG do Agente Aurora, descrevendo todas as etapas do processamento da Base Oficial de Conhecimento, desde a leitura dos documentos até a geração das respostas ao usuário. Foram definidos o papel do *loader*, a estratégia de fragmentação dos documentos, a geração de embeddings com Ollama, a utilização do FAISS como índice vetorial e a orquestração realizada pelo LangChain. Também foram estabelecidas boas práticas de implementação e critérios de qualidade para garantir precisão, modularidade e facilidade de evolução da solução.

ENCERRAMENTO DO BLOCO B

VOLUME 5

BLOCO C

ARQUITETURA, ENGENHARIA DE SOFTWARE E IMPLANTAÇÃO

CAPÍTULO 21

PRINCÍPIOS DE ENGENHARIA DE SOFTWARE

21.1 Objetivo

A implementação do Agente Aurora deverá seguir princípios de engenharia de software que privilegiem simplicidade, organização, reutilização e facilidade de manutenção.

Embora o projeto tenha sido desenvolvido como trabalho de conclusão da Formação Agentes de Inteligência Artificial da Alura, sua arquitetura foi planejada para permitir evolução futura sem necessidade de reescrita da aplicação.

21.2 Princípios adotados

O desenvolvimento será orientado pelos seguintes princípios:

- separação de responsabilidades;
 - baixo acoplamento entre módulos;
 - alta coesão;
 - configuração centralizada;
 - reutilização de código;
 - documentação contínua;
 - facilidade de testes;
 - arquitetura preparada para evolução.
-

CAPÍTULO 22

ORGANIZAÇÃO DO PROJETO

A estrutura oficial do repositório será:

aurora-ai-assistant/

|

└─ app.py

└─ requirements.txt

└─ README.md

|

```
└─ documentos/
|   Volume_1.pdf
|   Volume_2.pdf
|   Volume_3.pdf
|   Volume_4.pdf
|   Volume_5.pdf
|
└─ embeddings/
|   faiss_index/
|
└─ src/
|   loader.py
|   splitter.py
|   embeddings.py
|   rag.py
|   prompts.py
|   aurora.py
|   config.py
|
└─ images/
|
└─ tests/
```

Essa organização facilita a separação entre documentação, código-fonte, índices vetoriais e testes.

CAPÍTULO 23

RESPONSABILIDADE DOS MÓDULOS

app.py

Ponto de entrada da aplicação.

Responsável por:

- iniciar o Streamlit;
 - carregar os componentes principais;
 - controlar o fluxo da interface.
-

loader.py

Responsável pela leitura da Base Oficial.

Funções previstas:

- localizar PDFs;
 - carregar documentos;
 - extrair texto;
 - devolver objetos compatíveis com LangChain.
-

splitter.py

Responsável por:

- fragmentar documentos;
 - preservar contexto;
 - aplicar estratégia de chunking;
 - preparar os textos para geração dos embeddings.
-

embeddings.py

Responsável por:

- gerar embeddings;
 - persistir o índice FAISS;
 - recarregar índices existentes;
 - atualizar embeddings quando necessário.
-

rag.py

Núcleo do pipeline.

Responsabilidades:

- consultar FAISS;
 - recuperar contexto;
 - montar o prompt;
 - enviar consulta ao modelo.
-

prompts.py

Centralizará todos os prompts utilizados pelo Agente Aurora.

Essa separação evita que instruções fiquem espalhadas pelo código e facilita ajustes futuros.

aurora.py

Camada principal da aplicação.

Será responsável por:

- coordenar os módulos;
 - receber perguntas;
 - consultar o pipeline RAG;
 - devolver respostas formatadas.
-

config.py

Arquivo destinado às configurações globais.

Exemplos:

- modelo Ollama;
- diretórios;
- tamanho dos chunks;
- overlap;
- parâmetros do FAISS;
- configurações do Streamlit.

Toda configuração deverá ser concentrada neste módulo.

CAPÍTULO 24

PADRÕES DE DESENVOLVIMENTO

Durante a implementação deverão ser observadas as seguintes convenções.

Nomenclatura

Arquivos:

- minúsculos;
- nomes descritivos;
- separados por underscore apenas quando necessário.

Funções:

- verbos descritivos.

Exemplos:

`load_documents()`

`create_embeddings()`

build_vectorstore()

search_documents()

generate_answer()

Classes:

PascalCase.

Exemplo:

AuroraAssistant

VectorStoreManager

DocumentLoader

Variáveis:

snake_case.

CAPÍTULO 25

GERENCIAMENTO DE DEPENDÊNCIAS

Todas as bibliotecas utilizadas deverão ser registradas em:

requirements.txt

O arquivo permitirá reproduzir o ambiente em qualquer computador.

As dependências deverão permanecer organizadas e conter apenas os pacotes efetivamente utilizados pela aplicação.

CAPÍTULO 26

CONTROLE DE VERSÃO

Todo o desenvolvimento será versionado utilizando Git.

O repositório oficial deverá conter:

- código-fonte;
- documentação;
- imagens;

- README;
- instruções de instalação.

Arquivos temporários e índices gerados localmente poderão ser ignorados conforme política definida no .gitignore.

CAPÍTULO 27

DOCUMENTAÇÃO DO PROJETO

Além do código, deverão ser mantidos:

- README atualizado;
- instruções de instalação;
- descrição da arquitetura;
- explicação do pipeline RAG;
- tecnologias utilizadas;
- exemplos de utilização;
- imagens da interface.

Essa documentação facilitará a avaliação do projeto e sua manutenção futura.

CAPÍTULO 28

ESTRATÉGIA DE TESTES

Os testes deverão ocorrer em diferentes níveis.

Testes unitários

Verificar:

- leitura dos documentos;
 - fragmentação;
 - geração dos embeddings;
 - persistência do índice.
-

Testes de integração

Validar:

- comunicação entre Streamlit e LangChain;
 - recuperação vetorial;
 - geração das respostas.
-

Testes funcionais

Executar perguntas reais utilizando a Base Oficial.

Exemplos:

- "Quais acomodações estão disponíveis?"
 - "Qual é o horário do café da manhã?"
 - "A pousada aceita animais de estimação?"
 - "O que fazer em Campos do Jordão?"
-

CAPÍTULO 29

PREPARAÇÃO DO AMBIENTE

O desenvolvimento ocorrerá em duas etapas.

Google Colab

Utilizado para:

- estudo;
 - experimentação;
 - validação dos notebooks de referência;
 - testes iniciais do pipeline.
-

VS Code

Ambiente oficial de desenvolvimento.

Responsável por:

- implementação definitiva;
 - organização do projeto;
 - integração entre módulos;
 - preparação para deploy.
-

CAPÍTULO 30

PREPARAÇÃO PARA EVOLUÇÃO FUTURA

Embora a primeira versão utilize Ollama como modelo de linguagem, a arquitetura foi planejada para permitir futura substituição do provedor de IA.

A camada responsável pela comunicação com o modelo deverá permanecer isolada das demais partes da aplicação.

Com isso, será possível incorporar provedores como OpenAI, Gemini ou Claude sem necessidade de reestruturar o pipeline RAG, preservando a organização do código e a Base Oficial de Conhecimento.

Da mesma forma, o uso do FAISS como banco vetorial atende plenamente aos objetivos desta versão, mas sua abstração permitirá adoção de outras soluções em cenários futuros, caso haja necessidade de escalabilidade.

RESUMO EXECUTIVO DO BLOCO C

Este bloco estabelece os padrões de engenharia de software do Agente Aurora. Foram definidos a organização do repositório, as responsabilidades de cada módulo Python, as convenções de desenvolvimento, o gerenciamento de dependências, o controle de versão, a estratégia de documentação e os diferentes níveis de testes. Também foram estabelecidas diretrizes para preparação do ambiente de desenvolvimento e para a evolução futura da aplicação, garantindo que a arquitetura permaneça modular, organizada e preparada para incorporar novos modelos de linguagem e tecnologias sem comprometer a estrutura do projeto.

ENCERRAMENTO DO BLOCO C

VOLUME 5

BLOCO D

ARQUITETURA, ENGENHARIA DE SOFTWARE E IMPLANTAÇÃO

CAPÍTULO 31

ESTRATÉGIA DE IMPLANTAÇÃO

31.1 Objetivo

Após a conclusão do desenvolvimento local, o Agente Aurora será preparado para publicação na Oracle Cloud Infrastructure (OCI).

A implantação será realizada somente após a conclusão e validação de todos os testes funcionais, garantindo que a aplicação esteja estável antes de ser disponibilizada em ambiente de produção.

31.2 Ambientes do Projeto

O desenvolvimento será dividido em três ambientes distintos.

Ambiente de Prototipação

Ferramenta:

- Google Colab

Objetivos:

- estudo dos notebooks de referência;
 - experimentação;
 - validação do pipeline RAG;
 - testes rápidos.
-

Ambiente de Desenvolvimento

Ferramenta:

- Visual Studio Code

Objetivos:

- implementação definitiva;
 - organização modular;
 - integração entre componentes;
 - preparação do projeto.
-

Ambiente de Produção

Infraestrutura:

- Oracle Cloud Infrastructure (OCI)

Objetivos:

- disponibilizar a aplicação;
- permitir acesso ao Agente Aurora via navegador;
- validar a solução em ambiente semelhante ao real.

CAPÍTULO 32

ESTRATÉGIA DE DEPLOY

A implantação seguirá uma sequência controlada.

Etapas 1

Preparação do ambiente Python.

Etapas 2

Instalação das dependências.

Etapas 3

Configuração do Ollama.

Etapas 4

Geração do índice vetorial FAISS.

Etapas 5

Configuração da aplicação Streamlit.

Etapas 6

Testes locais.

Etapas 7

Publicação na Oracle Cloud Infrastructure.

Etapa 8

Validação pós-deploy.

CAPÍTULO 33

CRITÉRIOS DE ACEITAÇÃO

O projeto será considerado concluído quando atender aos seguintes critérios.

Base de Conhecimento

- cinco volumes consolidados;
 - PDFs revisados;
 - documentação padronizada;
 - estrutura consistente.
-

Pipeline

- leitura correta dos documentos;
 - chunking executado;
 - embeddings gerados;
 - índice FAISS criado.
-

Agente Aurora

O sistema deverá responder corretamente perguntas relacionadas a:

- pousada;
 - acomodações;
 - serviços;
 - políticas;
 - turismo;
 - perguntas frequentes.
-

Interface

A interface Streamlit deverá:

- permitir interação por chat;
- apresentar respostas em linguagem natural;
- manter histórico da conversa durante a sessão;
- possuir navegação simples.

Código

O projeto deverá apresentar:

- organização modular;
- documentação adequada;
- comentários quando necessários;
- estrutura compatível com manutenção futura.

CAPÍTULO 34

ORGANIZAÇÃO DO REPOSITÓRIO

O repositório oficial deverá conter:

aurora-ai-assistant/

└─ app.py

└─ requirements.txt

└─ README.md

|

└─ documentos/

|

└─ embeddings/

|

└─ src/

|

└─ testes/

|

└─ imagens/

|

└─ .gitignore

README

O README deverá conter:

- descrição do projeto;
- objetivo;
- arquitetura;

- tecnologias utilizadas;
 - instruções de instalação;
 - modo de execução;
 - capturas de tela;
 - referências utilizadas.
-

CAPÍTULO 35

ROADMAP DE EVOLUÇÃO

Embora esta versão tenha sido concebida para atender aos requisitos do desafio, sua arquitetura permite evolução contínua.

Entre as possibilidades futuras destacam-se:

Inteligência Artificial

- substituição do Ollama por OpenAI;
 - integração com Google Gemini;
 - integração com Claude;
 - comparação entre diferentes modelos de linguagem.
-

Banco Vetorial

Possível substituição do FAISS por soluções mais robustas, caso o volume de documentos ou o número de usuários justifique essa evolução.

Funcionalidades

Possíveis expansões:

- consulta de disponibilidade em tempo real;
- reservas online;
- painel administrativo;
- área de gerenciamento da Base Oficial de Conhecimento;
- autenticação de usuários;
- métricas de utilização;
- suporte multilíngue;
- recomendações personalizadas.

Essas evoluções deverão preservar a arquitetura modular estabelecida neste documento.

CAPÍTULO 36

BOAS PRÁTICAS PARA EVOLUÇÃO DO PROJETO

Toda evolução futura deverá observar os seguintes princípios:

- manter a Base Oficial de Conhecimento como fonte única de informação;
 - preservar a separação entre documentação e código;
 - evitar dependências desnecessárias;
 - manter baixo acoplamento entre módulos;
 - centralizar configurações;
 - documentar alterações arquiteturais;
 - revisar a Base Oficial sempre que novos conteúdos forem incorporados.
-

CAPÍTULO 37

CONSIDERAÇÕES FINAIS

O Agente Aurora representa a integração entre engenharia de software, inteligência artificial e gestão do conhecimento.

Ao longo deste projeto foram produzidos cinco volumes que documentam tanto o domínio da aplicação quanto sua arquitetura técnica, formando uma Base Oficial de Conhecimento estruturada para utilização em sistemas Retrieval-Augmented Generation (RAG).

A solução foi concebida com foco em simplicidade, organização e possibilidade de evolução, permitindo que novas funcionalidades sejam incorporadas sem comprometer os princípios arquiteturais definidos.

Além de atender aos objetivos do desafio, a arquitetura estabelece uma fundação sólida para futuras pesquisas e aplicações envolvendo agentes inteligentes, processamento de linguagem natural e recuperação semântica de informações.

CAPÍTULO 38

TERMO DE ENCERRAMENTO DA DOCUMENTAÇÃO TÉCNICA

Com a conclusão deste volume, considera-se encerrada a fase de documentação do projeto **Agente Aurora**.

A documentação produzida compreende:

- Base Oficial de Conhecimento;
- arquitetura funcional;
- arquitetura técnica;
- pipeline RAG;
- organização do software;

- critérios de desenvolvimento;
- diretrizes para implantação.

Esses documentos constituem a referência oficial para todas as etapas subsequentes de implementação.

CAPÍTULO 39

PRÓXIMA ETAPA DO PROJETO

Com a documentação consolidada, inicia-se a fase de engenharia de software, seguindo o cronograma estabelecido para o desafio.

As atividades serão desenvolvidas na seguinte ordem:

Etapa 1 — Preparação do Ambiente

- configuração do ambiente Python;
 - organização do repositório;
 - instalação das dependências;
 - validação dos notebooks de referência.
-

Etapa 2 — Construção da Base Vetorial

- leitura dos cinco volumes;
 - fragmentação dos documentos;
 - geração dos embeddings;
 - criação do índice FAISS.
-

Etapa 3 — Implementação do Pipeline RAG

- integração com LangChain;
 - recuperação semântica;
 - composição do contexto;
 - geração das respostas pelo modelo Ollama.
-

Etapa 4 — Desenvolvimento da Interface

- construção da aplicação em Streamlit;
 - integração com o Agente Aurora;
 - testes de interação.
-

Etapa 5 — Testes e Publicação

- testes funcionais;
 - validação da experiência do usuário;
 - preparação do repositório GitHub;
 - implantação na Oracle Cloud Infrastructure (OCI).
-

CAPÍTULO 40

DECLARAÇÃO DE CONFORMIDADE

Este documento consolida a arquitetura oficial do **Agente Aurora** e registra as decisões técnicas adotadas para o projeto de conclusão da Formação Agentes de Inteligência Artificial da Alura.

A implementação deverá observar os seguintes princípios:

- utilização dos **notebooks do Google Colab como referência técnica** para o desenvolvimento do código em Python;
- desenvolvimento inicial em ambiente local;
- adoção de **Python, LangChain, FAISS, Ollama e Streamlit** como tecnologias principais;
- utilização da **Base Oficial de Conhecimento (Volumes 1 a 5)** como única fonte autorizada para o pipeline RAG;
- preparação da arquitetura para futura substituição do modelo de linguagem sem necessidade de reescrever a aplicação;
- implantação final na Oracle Cloud Infrastructure (OCI).

Qualquer evolução futura deverá preservar a modularidade da arquitetura e a separação entre conhecimento, lógica de aplicação e interface.

RESUMO EXECUTIVO DO VOLUME 5

O Volume 5 – Revisão Final consolida toda a arquitetura técnica do Agente Aurora. Foram definidos os princípios de engenharia de software, a arquitetura oficial da solução, o pipeline RAG, a organização do projeto, os padrões de desenvolvimento, a estratégia de testes, o plano de implantação na Oracle Cloud Infrastructure e o roadmap de evolução. Esta revisão substitui integralmente a versão anterior, eliminando referências a tecnologias que não fazem parte do escopo do desafio e alinhando a documentação ao stack oficial composto por **Python, Google Colab, LangChain, Ollama, FAISS, Streamlit e OCI**.

ENCERRAMENTO OFICIAL DA DOCUMENTAÇÃO